ADMINISTRACIÓN BASE DE DATOS

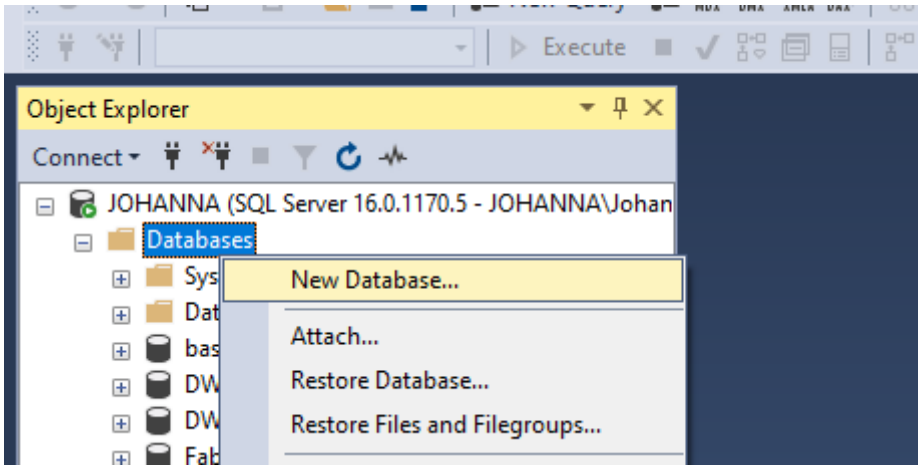
PRÁCTICA GUIADA 1

Temas: Transacciones, Propiedades ACID, Bloqueos y Niveles de Aislamiento.

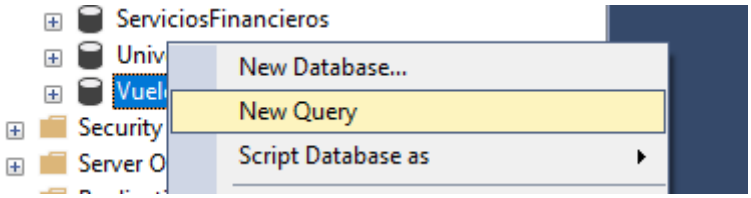
Bloque 1: Crear la base de datos y registros

La base de datos tiene por objetivo guardar vuelos y registrar reservas en dichos vuelos. Se cuenta con un esquema básico de 2 tablas.

Crear la base de datos Vuelos



Una vez creada, clic derecho New Query



Y copiamos y pegamos el siguiente script

```
-- Crear Tablas Base
CREATE TABLE Vuelos (
   vuelo_id INT PRIMARY KEY,
   destino VARCHAR(50),
   asientos_disponibles INT,
   precio DECIMAL(10,2)
);


CREATE TABLE Reservas (
```

```
  reserva_id INT IDENTITY(1,1) PRIMARY KEY,
  vuelo_id INT,
  pasajero_nombre VARCHAR(100),
  fecha_reserva DATETIME DEFAULT GETDATE(),
  FOREIGN KEY (vuelo_id) REFERENCES Vuelos(vuelo_id)
);
-- Insertar datos de prueba
INSERT INTO Vuelos VALUES (101, 'Galápagos', 2, 150.00), (102, 'Quito', 15, 60.00);
```

Conclusión:

El script va a permitir generar la tabla padre Vuelos y la tabla hija Reservas, con sus respectivas claves, además se insertan dos vuelos con los códigos 101 y 102.

Bloque 2: Transacciones y Propiedades ACID

```
--Reservar un asiento en el vuelo 101
BEGIN TRANSACTION;
BEGIN TRY
INSERT INTO Reservas (vuelo_id, pasajero_nombre)
VALUES (101, 'Carlos Andrade');
 UPDATE Vuelos
  SET asientos_disponibles = asientos_disponibles - 1
  WHERE vuelo_id = 101;
IF (SELECT asientos_disponibles FROM Vuelos WHERE vuelo_id = 101) < 0
    THROW 50000, 'No hay asientos disponibles.', 1;
COMMIT;
PRINT 'Reserva realizada con exito';
END TRY
BEGIN CATCH
ROLLBACK;
PRINT 'Error: '+ERROR_MESSAGE();
```

END CATCH;

--Comprobemos la transacción

*SELECT *FROM Reservas WHERE vuelo_id=101;*

*SELECT *FROM Vuelos*

Conclusión: En esta transacción básica están presentes todas las propiedades ACID de una transacción, se utiliza try para garantizar que se ejecuten los bloques correspondiente que den como resultado una transacción exitosa y en el caso del catch para el bloque de sentencias cuando la transacción falle por algún motivo y se tenga que deshacer toda la transacción sin que la base de datos se vea afectada.

Bloque 3: Simulación de bloqueos

Simulación 1.

Aquí se demostrará cómo el motor de base de datos protege la Integridad y evita que dos personas reserven el mismo asiento al mismo tiempo.

Abrimos una query y escribimos el siguiente script.

BEGIN TRANSACTION;

UPDATE Vuelos

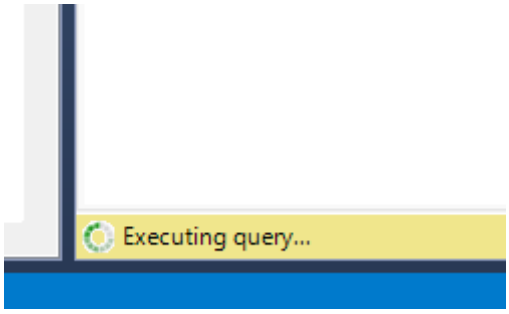
SET asientos_disponibles = asientos_disponibles - 1

WHERE vuelo_id = 101;

Abrimos una segunda query y escribirmos el script.

*SELECT * FROM Vuelos WHERE vuelo_id = 101;*

Observemos:



Volvemos al primer query y escribimos:

COMMIT;

Conclusión:

El momento en que la ventana del segundo query se quedó "congelada" es la prueba real de que el motor de base de datos está trabajando para mantener la integridad. Únicamente se desbloqueó cuando hicimos un COMMIT, es decir completamos la transacción.

Simulación 2.

Abrimos un query y escribimos el siguiente script

BEGIN TRANSACTION;

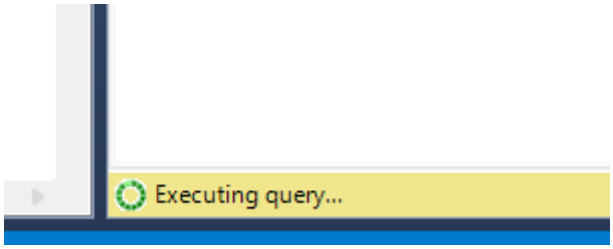
UPDATE Vuelos

SET asientos_disponibles = asientos_disponibles - 1

WHERE vuelo_id = 101;

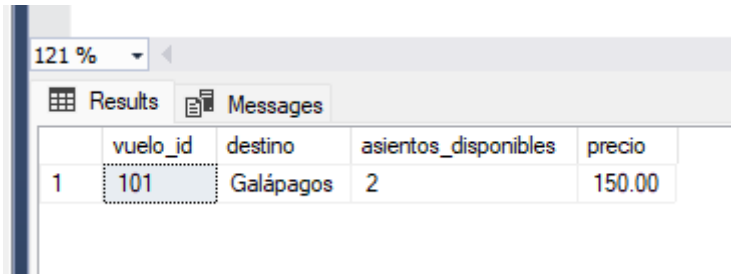
Luego abrimos un nuevo query y escribimos:

*SELECT * FROM Vuelos WHERE vuelo_id = 101;*



Volver al query inicial y escribir:

ROLLBACK;



	vuelo_id	destino	asientos_disponibles	precio
1	101	Galápagos	2	150.00

Conclusión.

La ventana del segundo query se desbloqueó, pero el asiento no se restó. El ROLLBACK es como viajar al pasado; deshizo el error antes de que alguien más viera datos incorrectos.

Ahora bien, un DBA no puede estar dando clic a COMMIT ó ROLLBACK manualmente. Hay que automatizarlo.

Aquí se presenta un código automatizado para el control de bloqueos.

```
BEGIN TRANSACTION;
BEGIN TRY
  -- Intentamos restar el asiento
  UPDATE Vuelos SET asientos_disponibles = asientos_disponibles - 1 WHERE vuelo_id = 101;

  -- Si los asientos son menores a 0, forzamos un error
  IF (SELECT asientos_disponibles FROM Vuelos WHERE vuelo_id = 101) < 0
    THROW 50000, 'Error: No hay más asientos.', 1;

  COMMIT; -- Si todo está bien, guardamos
  PRINT 'Reserva exitosa';
END TRY
BEGIN CATCH
  ROLLBACK; -- Si algo falló (como el THROW), deshacemos todo
  PRINT 'Transacción cancelada: ' + ERROR_MESSAGE();
END CATCH
```

Conclusión:

Commit vs. Rollback actúan como el guardar definitivo en un documento en Word y el ctrl+z, con Commit mandamos guardar de forma permanente la transacción en la base de datos y con ROLLBACK finalizamos la transacción deshaciendo los cambios que se hicieron para que la base de datos no se vea afectada.

Bloque 4: Niveles de aislamiento

Los niveles de aislamiento definen el grado en que una transacción está protegida, son trabajo estricto del administrador de base de datos.

Simulación 1. READ UNCOMMITTED, pueden ocurrir lecturas de datos de transacciones no confirmadas.

En una ventana query ejecuta esto para dejar la transacción "a medias":

```
BEGIN TRANSACTION;
UPDATE Vuelos SET asientos_disponibles = 50 WHERE vuelo_id = 101;
```

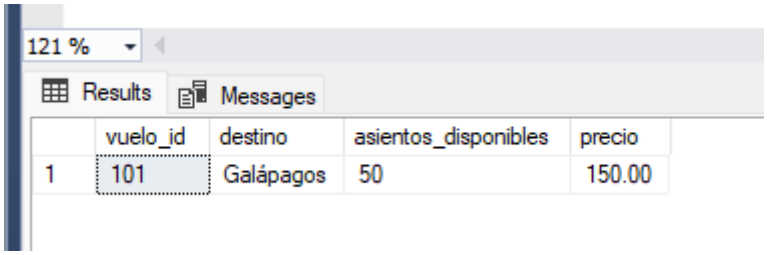
NO EJECUTAR COMMIT NI ROLLBACK AÚN

En una segunda ventana query ejecuta esto para forzar al sistema a que no respete bloqueos:

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

*SELECT * FROM Vuelos WHERE vuelo_id = 101;*

Deberás ver esto:



	vuelo_id	destino	asientos_disponibles	precio
1	101	Galápagos	50	150.00

Conclusión:

La segunda ventana del query está viendo un cambio que la primer ventana query todavía no confirma. Si en la ventana del primer query se hace ROLLBACK, la segunda ventana a la larga leyó información falsa, la denominada lectura sucia.

Simulación 2, READ COMMITED, protege la integridad obligando a esperar. evita lecturas de datos de transacciones no confirmadas.

En una ventana query ejecuta esto para dejar la transacción "a medias":

BEGIN TRANSACTION;

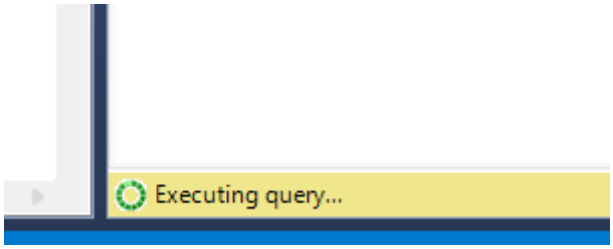
UPDATE Vuelos SET asientos_disponibles = 50 WHERE vuelo_id = 101;

NO EJECUTAR COMMIT NI ROLLBACK AÚN

En una segunda ventana query cambia el nivel al que viene por defecto en SQL Server:

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

*SELECT * FROM Vuelos WHERE vuelo_id = 101;*



Resultado: La ventana se queda "congelada" Executing query

Conclusión:

Noten la diferencia la segunda ventana query no acepta mentiras. Como la primera ventana tiene el dato 'secuestrado', la ventana 2 del query prefiere esperar a que sea oficial antes de mostrarlo.

Simulación 3, REPETEABLE READ, asegura consistencia en las lecturas, aunque pueden existir anomalías con inserciones.

Para probar este nivel de aislamiento, insertamos una nueva fila en la tabla Vuelos.

INSERT INTO Vuelos VALUES (103, 'Miami', 3, 200.00);

Abrimos una primera ventana query a la que llamaremos ventana A, el REPEATABLE READ pondrá un candado sobre la fila del vuelo 103.

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

BEGIN TRANSACTION;

SELECT * FROM Vuelos WHERE vuelo_id= 103;

En una segunda ventana query, ventana B, intentamos cambiar el valor del vuelo como administradores.

UPDATE Vuelos SET precio = 350.00 WHERE vuelo_id = 103;

La consulta se quedará cargando. El motor de la base de datos está obligando a esta ventana a esperar porque la Ventana A tiene la fila protegida o bloqueada.

Volvemos a la Ventana A.

*SELECT * FROM Vuelos WHERE vuelo_id= 103;*

Resultado: Sigue siendo 200.00

Entonces comprobamos la Consistencia y el Aislamiento. Nada ha cambiado para nosotros como admin.

Ahora estando en la misma ventana A del query, terminamos la transacción con un COMMIT.

COMMIT;

En cuanto des clic en COMMIT en la Ventana A, verás que la Ventana B (que estaba cargando o aparecía el Executing Query) termina de golpe. El candado se liberó y el precio finalmente se actualizó.

Cuando usas este nivel, ocurre lo siguiente:

- Al leer el dato, pones un candado compartido.
- Si otra persona intenta hacer un UPDATE, el motor de la base de datos le dice: Espera, alguien está leyendo esto y tiene prohibido que cambie.
- El candado solo se suelta cuando tú haces COMMIT o ROLLBACK.

Simulación 4. Nivel SERIALIZABLE, aquí no solo se bloquean filas existentes, sino que se "congela" la tabla para que nadie inserte nada nuevo que altere el resultado.

En una ventana query, escribimos.

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRANSACTION;

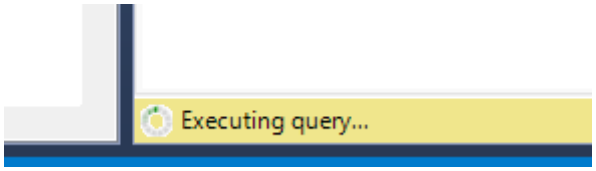
*SELECT * FROM Vuelos WHERE precio < 100;*

-No cierres aún la transacción.

En una segunda ventana query intentamos insertar un vuelo nuevo que cueste $50.

INSERT INTO Vuelos VALUES (103, 'Cuenca', 20, 50.00);

Como resultado, se bloquea.



Conclusión.

En el nivel SERIALIZABLE, SQL dice, no me toques nada que tenga un precio menor a 100, porque estoy haciendo un reporte y no quiero que aparezcan registros fantasmas de la nada. Es el nivel de mayor protección.